

day2 讲义

Equifix

2026 年 4 月 30 日

删除实体

题意：给定若干个点以及若干个高为 1 的长方形，所有点以相同速率下落，若某个长方形与点接触，则立即删除自身以及和自身接触的所有点，问每个点是在什么高度被删除的。

$n, m \leq 10^5$ 。

删除实体

- 无法理解这题作为一道 icpc 题为什么过的队数的对数为 0。

删除实体

- 无法理解这题作为一道 icpc 题为什么过的队数的对数为 0。
- x 轴每个点开个栈，从低到高扫描，每次遇到一个长方形就直接在 x 轴对应区间的栈 push 自身，遇到若干个点就查询这些位置的栈顶分别是谁，然后懒标记弹栈即可。

删除实体

- 无法理解这题作为一道 icpc 题为什么过的队数的对数为 0。
- x 轴每个点开个栈，从低到高扫描，每次遇到一个长方形就直接在 x 轴对应区间的栈 push 自身，遇到若干个点就查询这些位置的栈顶分别是谁，然后懒标记弹栈即可。
- 维护的话，首先离散化，然后开一棵线段树，每个节点开一个栈，每次遇到长方形就直接在该区间对应的那 $O(\log n)$ 个节点 push，查询的时候直接一路走到底，每次查询栈顶的纵坐标，最后取纵坐标最大的那个就行了。

删除实体

- 无法理解这题作为一道 icpc 题为什么过的队数的对数为 0。
- x 轴每个点开个栈，从低到高扫描，每次遇到一个长方形就直接在 x 轴对应区间的栈 push 自身，遇到若干个点就查询这些位置的栈顶分别是谁，然后懒标记弹栈即可。
- 维护的话，首先离散化，然后开一棵线段树，每个节点开一个栈，每次遇到长方形就直接在该区间对应的那 $O(\log n)$ 个节点 push，查询的时候直接一路走到底，每次查询栈顶的纵坐标，最后取纵坐标最大的那个就行了。
- 同一高度的点查询完之后，再懒标记一下哪些长方形被删除了即可。

删除实体

- 无法理解这题作为一道 icpc 题为什么过的队数的对数为 0。
- x 轴每个点开个栈，从低到高扫描，每次遇到一个长方形就直接在 x 轴对应区间的栈 push 自身，遇到若干个点就查询这些位置的栈顶分别是谁，然后懒标记弹栈即可。
- 维护的话，首先离散化，然后开一棵线段树，每个节点开一个栈，每次遇到长方形就直接在该区间对应的那 $O(\log n)$ 个节点 push，查询的时候直接一路走到底，每次查询栈顶的纵坐标，最后取纵坐标最大的那个就行了。
- 同一高度的点查询完之后，再懒标记一下哪些长方形被删除了即可。
- 时间复杂度 $O(n \log n)$ 。

题意：让你构造一个长为 $3n$ 的 01 串， q 个要求，要求要么区间 $[a_i, b_i]$ 全是 0，要么区间 $[c_i, d_i]$ 全是 1，且序列最多 $2n$ 个 0 和 $2n$ 个 1。
 $n \leq 3 \times 10^4, q \leq 10^5$ ，多测， $\sum n \leq 3 \times 10^5, \sum q \leq 10^6$ 。

- 一股 2-SAT 味, $[a_i, b_i]$ 中任何一个位置填 1 即可推出 $[c_i, d_i]$ 全为 1, 反之亦然。

萝卜内存

- 一股 2-SAT 味, $[a_i, b_i]$ 中任何一个位置填 1 即可推出 $[c_i, d_i]$ 全为 1, 反之亦然。
- 难点在于构造, 注意到这里的限制是 1 能推出 1, 所以我们考虑只保留这半边的点和边, 来看一下。

- 一股 2-SAT 味, $[a_i, b_i]$ 中任何一个位置填 1 即可推出 $[c_i, d_i]$ 全为 1, 反之亦然。
- 难点在于构造, 注意到这里的限制是 1 能推出 1, 所以我们考虑只保留这半边的点和边, 来看一下。
- 使用线段树优化建图 (可以开两个线段树, 一个指向点一个从点指出), tarjin 缩一下点, 这样我们如果要选一个点为 1, 那么它能到达的所有点都必须得是 1; 如果要选一个点为 0, 考虑反图上这个点能到达的点在正图上就是能到达这个点的点, 这样的点就必须是 0。

- 一股 2-SAT 味, $[a_i, b_i]$ 中任何一个位置填 1 即可推出 $[c_i, d_i]$ 全为 1, 反之亦然。
- 难点在于构造, 注意到这里的限制是 1 能推出 1, 所以我们考虑只保留这半边的点和边, 来看一下。
- 使用线段树优化建图 (可以开两个线段树, 一个指向点一个从点指出), tarjin 缩一下点, 这样我们如果要选一个点为 1, 那么它能到达的所有点都必须得是 1; 如果要选一个点为 0, 考虑反图上这个点能到达的点在正图上就是能到达这个点的点, 这样的点就必须是 0。
- 线段树优化建图需要注意细节, 不能无脑将一个区间的 $O(\log n)$ 条边与另一边的 $O(\log n)$ 条边进行连接, 中间新建一个点, 出边连向这个点, 入边从这个点引出即可, 不然复杂度多个 $\log$ 可能会 TLE。

- 然后是构造的问题。

- 然后是构造的问题。
- 如果所有强连通块大小都小于等于 n ，那我们可以直接按照拓扑序来赋值，一段前缀赋值成 0，这样就能保证每个点被赋值成 0 当且仅当这个点的所有入边那头的点都被赋值成了 0。这样我们一定可以构造出 0 的个数在 $[n, 2n]$ 这个区间里的解。

- 然后是构造的问题。
- 如果所有强连通块大小都小于等于 n ，那我们可以直接按照拓扑序来赋值，一段前缀赋值成 0，这样就能保证每个点被赋值成 0 当且仅当这个点的所有入边那头的点都被赋值成了 0。这样我们一定可以构造出 0 的个数在 $[n, 2n]$ 这个区间里的解。
- 如果存在一个强连通块大小超过 n ，我们直接无脑将这个强连通块赋值为 1，并将这个强连通块能到达的所有强连通块都赋值成 1。

- 然后是构造的问题。
- 如果所有强连通块大小都小于等于 n ，那我们可以直接按照拓扑序来赋值，一段前缀赋值成 0，这样就能保证每个点被赋值成 0 当且仅当这个点的所有入边那头的点都被赋值成了 0。这样我们一定可以构造出 0 的个数在 $[n, 2n]$ 这个区间里的解。
- 如果存在一个强连通块大小超过 n ，我们直接无脑将这个强连通块赋值为 1，并将这个强连通块能到达的所有强连通块都赋值成 1。
- 这样如果 1 的个数不超过 $2n$ ，直接就是解了；否则，我们重新考虑将这个连通块和能到达这个连通块的所有点都赋为 0，其余点赋为 1，若 0 的个数超过 $2n$ ，那一定就无解了（因为这个连通块赋值为什么都不符合条件），不超过 $2n$ 那我们就找到了解。

萝卜内存

- 然后是构造的问题。
- 如果所有强连通块大小都小于等于 n ，那我们可以直接按照拓扑序来赋值，一段前缀赋值成 0，这样就能保证每个点被赋值成 0 当且仅当这个点的所有入边那头的点都被赋值成了 0。这样我们一定可以构造出 0 的个数在 $[n, 2n]$ 这个区间里的解。
- 如果存在一个强连通块大小超过 n ，我们直接无脑将这个强连通块赋值为 1，并将这个强连通块能到达的所有强连通块都赋值成 1。
- 这样如果 1 的个数不超过 $2n$ ，直接就是解了；否则，我们重新考虑将这个连通块和能到达这个连通块的所有点都赋为 0，其余点赋为 1，若 0 的个数超过 $2n$ ，那一定就无解了（因为这个连通块赋值为什么都不符合条件），不超过 $2n$ 那我们就找到了解。
- 做完了。时间复杂度是优化建图和 tarjin 的 $O(n + q \log n)$ 。

题意：给你一棵树，让你对所有 $k \leq \lfloor \frac{n}{2} \rfloor$ 求 k 对点距离和最大值。
 $n \leq 10^6$ 。

- 暴力埋了。考虑这题的几个关键结论。

- 暴力埋了。考虑这题的几个关键结论。
- 首先是如果我们不考虑配对，只考虑每一个 k 的答案的点集 S_k 的话，有 $S_k \subset S_{k+1}$ 对所有 k 成立。

遥远恋情

- 暴力埋了。考虑这题的几个关键结论。
- 首先是如果我们不考虑配对，只考虑每一个 k 的答案的点集 S_k 的话，有 $S_k \subset S_{k+1}$ 对所有 k 成立。
- 这个的证明可以考虑模拟费用流的流程，这题的费用流建模还是比较简单的。

- 暴力埋了。考虑这题的几个关键结论。
- 首先是如果我们不考虑配对，只考虑每一个 k 的答案的点集 S_k 的话，有 $S_k \subset S_{k+1}$ 对所有 k 成立。
- 这个的证明可以考虑模拟费用流的流程，这题的费用流建模还是比较简单的。
- 然后是，如果我们确定了点集 S_k ，该如何计算距离和。这个是很简单的，首先注意到所有路径一定至少都经过了一个点，原因是如果存在两条路径不交，那么交换一下一端就可以得到更优的配对。注意到如果将 S_k 内的点视为点权为 1，其余点视为点权为 0，则同时经过的点必为带权重心之一，这个是很显然的，而路径和可以被转化为每个点到重心的距离和。

- 暴力埋了。考虑这题的几个关键结论。
- 首先是如果我们不考虑配对，只考虑每一个 k 的答案的点集 S_k 的话，有 $S_k \subset S_{k+1}$ 对所有 k 成立。
- 这个的证明可以考虑模拟费用流的流程，这题的费用流建模还是比较简单的。
- 然后是，如果我们确定了点集 S_k ，该如何计算距离和。这个是很简单的，首先注意到所有路径一定至少都经过了一个点，原因是如果存在两条路径不交，那么交换下一端就可以得到更优的配对。注意到如果将 S_k 内的点视为点权为 1，其余点视为点权为 0，则同时经过的点必为带权重心之一，这个是很显然的，而路径和可以被转化为每个点到重心的距离和。
- 接下来就是我们该如何确定点集的事情。

遥远恋情

- 考虑从 $S_0 = \emptyset$ 开始逐渐加点。

遥远恋情

- 考虑从 $S_0 = \emptyset$ 开始逐渐加点。
- 对于 S_k , 重心 c 可能的取值范围一定是一条链, 这条链也就是所有路径的交。如果链没有退化, 那么 S_k 中的所有点一定分布在这条链的两端, 且两端的点个数恰为 k 。如果我们首先将集合中加入一点 A , 其在这条链上的投影为 a , 则重心会立即移动至 a 处; 如果我们再加入一点 B , 其投影为 b 的话, 那么重心的取值范围将变为 $a \sim b$ 这条链。

遥远恋情

- 考虑从 $S_0 = \emptyset$ 开始逐渐加点。
- 对于 S_k , 重心 c 可能的取值范围一定是一条链, 这条链也就是所有路径的交。如果链没有退化, 那么 S_k 中的所有点一定分布在这条链的两端, 且两端的点个数恰为 k 。如果我们首先将集合中加入一点 A , 其在这条链上的投影为 a , 则重心会立即移动至 a 处; 如果我们再加入一点 B , 其投影为 b 的话, 那么重心的取值范围将变为 $a \sim b$ 这条链。
- 如果每次选择离当前重心最远的点 A , 执行两次操作后我们断言新集合就是 S_{k+1} 。

遥远恋情

- 考虑从 $S_0 = \emptyset$ 开始逐渐加点。
- 对于 S_k , 重心 c 可能的取值范围一定是一条链, 这条链也就是所有路径的交。如果链没有退化, 那么 S_k 中的所有点一定分布在这条链的两端, 且两端的点个数恰为 k 。如果我们首先将集合中加入一点 A , 其在这条链上的投影为 a , 则重心会立即移动至 a 处; 如果我们再加入一点 B , 其投影为 b 的话, 那么重心的取值范围将变为 $a \sim b$ 这条链。
- 如果每次选择离当前重心最远的点 A , 执行两次操作后我们断言新集合就是 S_{k+1} 。
- 证明比较容易: 一开始我们一定选择一条直径, 故整棵树的重心, 以及剩余子树也就是 $V \setminus S_k$ 的重心一定在 c 的取值链里; 之后我们的操作要么固定 c , 要么加入一条 $V \setminus S_k$ 的直径: 先找到任意一点的距离最远的点, 再找到重心距离最远的点; 若找到和第一次找到的点在相同子树的点, 设先后找到的点是 A, B , 则我们将原本 S_k 中的点取出一个配对好的, 将它们的终点设为 A 和 B , 这样 c 就固定了, 且简单讨论一下发现 c 一定是原树重心; 要么找到的是不同子树的点, 这时候就是加入了一条新直径。

- 维护这整个过程却显得相当繁琐。

- 维护这整个过程却显得相当繁琐。
- 我们需要维护当前重心 c ，需要一种数据结构支持查询当前点集内的点到 c 的距离和，需要支持查询点集外的点到 c 的最大距离。

- 维护这整个过程却显得相当繁琐。
- 我们需要维护当前重心 c ，需要一种数据结构支持查询当前点集内的点到 c 的距离和，需要支持查询点集外的点到 c 的最大距离。
- 距离和可以用点分树简单维护；最大距离也可以使用点分树，每一层维护本层所有点集外的点到根的最大距离，朴素实现可以用 set 或优先队列等方式维护，当然由于点集外的点只删不加，可以提前将该层所有点按到根的距离排序，之后只需要维护一个单向移动的指针就行，排序可以基数排序，这样可以少个 $\log$ 。

- 维护这整个过程却显得相当繁琐。
- 我们需要维护当前重心 c ，需要一种数据结构支持查询当前点集内的点到 c 的距离和，需要支持查询点集外的点到 c 的最大距离。
- 距离和可以用点分树简单维护；最大距离也可以使用点分树，每一层维护本层所有点集外的点到根的最大距离，朴素实现可以用 set 或优先队列等方式维护，当然由于点集外的点只删不加，可以提前将该层所有点按到根的距离排序，之后只需要维护一个单向移动的指针就行，排序可以基数排序，这样可以少个 $\log$ 。
- 求重心 c 有一个小 trick，我们按 dfs 序将点集内的点排好，我们知道重心 c 一定在第 $\lfloor \frac{m}{2} \rfloor + 1$ 个点到 1 的路径上，因为最浅重心的父亲必然满足 $siz > \lfloor \frac{m}{2} \rfloor$ ，根据抽屉原理和 dfs 序性质，其子树必然包含这个点。然后可以利用重剖、倍增等方式维护。

遥远恋情

- 维护这整个过程却显得相当繁琐。
- 我们需要维护当前重心 c ，需要一种数据结构支持查询当前点集内的点到 c 的距离和，需要支持查询点集外的点到 c 的最大距离。
- 距离和可以用点分树简单维护；最大距离也可以使用点分树，每一层维护本层所有点集外的点到根的最大距离，朴素实现可以用 set 或优先队列等方式维护，当然由于点集外的点只删不加，可以提前将该层所有点按到根的距离排序，之后只需要维护一个单向移动的指针就行，排序可以基数排序，这样可以少个 $\log$ 。
- 求重心 c 有一个小 trick，我们按 dfs 序将点集内的点排好，我们知道重心 c 一定在第 $\lfloor \frac{m}{2} \rfloor + 1$ 个点到 1 的路径上，因为最浅重心的父亲必然满足 $siz > \lfloor \frac{m}{2} \rfloor$ ，根据抽屉原理和 dfs 序性质，其子树必然包含这个点。然后可以利用重剖、倍增等方式维护。
- 总之这个方法相当麻烦，且常数大，不知道有没有大手子写了，期望是过不了的。

- 加点难，那删点是不是容易呢？

- 加点难，那删点是不是容易呢？
- 删点的结论与加点类似。我们从全集开始，每次删除离当前重心最近的那个点，当剩余点数为偶数时这就是答案点集。证明基本对称但略有区别，这里略过。

- 加点难，那删点是不是容易呢？
- 删点的结论与加点类似。我们从全集开始，每次删除离当前重心最近的那个点，当剩余点数为偶数时这就是答案点集。证明基本对称但略有区别，这里略过。
- 考虑这给我们从维护的角度讲带来了什么便利。首先是，当删除一个点后，如果点集剩余偶数个点，那重心根本不需要动；如果剩余奇数个点，则我们可以通过哪个子树过大来确定重心移动的方向。按 1 为根 dfs 一下，设当前重心需要往子树方向移动，则按照前文方法，找到 dfs 序中位数，用倍增跳到重心下面，随后，使用子树内 dfs 序最小的点和最大的点，直接求 LCA 就是新重心的位置；否则，重心需要往父亲方向移动，依旧是利用 LCA 来求，这时分类一下左半边和右半边是否有点，找到外部子树离当前重心最近的点挪过去就行了。

- 我们同时需要维护当前点集内离重心最近的点是谁。此时我们发现，LCA 移动的过程中，经过的那些点下面一定没有属于自己的点集内点！故我们可以完完全全忽略掉这个部分，直接利用线段树做两次区间加操作，就可以维护出所有点的距离，这样维护距离可轻松太多了。

- 我们同时需要维护当前点集内离重心最近的点是谁。此时我们发现，LCA 移动的过程中，经过的那些点下面一定没有属于自己的点集内点！故我们可以完完全全忽略掉这个部分，直接利用线段树做两次区间加操作，就可以维护出所有点的距离，这样维护距离可轻松太多了。
- 这样我们就可以解决此题了，时间复杂度为小常数的 $O(n \log n)$ 。

题意：给定一个长度为 n 的数组，你需要进行恰好 k 次操作，每次选择两个数 a_i 和 a_j ($i \neq j$)，将这两个数从数组中删去并将他们的最大公约数加入数组中。最大化操作结束之后数组的总和。
 $n \leq 10^6$ ，值域 $m \leq 10^{18}$ ，简单版本 $m \leq 10^6$ 。

- 诡异。gcd 居然和 sum 扯上关系了。

- 诡异。gcd 居然和 sum 扯上关系了。
- 不难发现如果我们把 k 次操作综合起来看的话，我们实际上是将若干个集合的数合并成了一个公共 gcd。如果真的有很多很多个集合的话，这个问题就太难了，因为我们不仅要考虑怎么去选，还要考虑分开考虑每个集合的时候会不会选到相同的数。

- 诡异。gcd 居然和 sum 扯上关系了。
- 不难发现如果我们把 k 次操作综合起来看的话，我们实际上是将若干个集合的数合并成了一个公共 gcd。如果真的有很多很多个集合的话，这个问题就太难了，因为我们不仅要考虑怎么去选，还要考虑分开考虑每个集合的时候会不会选到相同的数。
- 我们猜一下结论，是不是我们这 k 次操作只合并了同一个集合。

- 诡异。gcd 居然和 sum 扯上关系了。
- 不难发现如果我们把 k 次操作综合起来看的话，我们实际上是将若干个集合的数合并成了一个公共 gcd。如果真的有很多很多个集合的话，这个问题就太难了，因为我们不仅要考虑怎么去选，还要考虑分开考虑每个集合的时候会不会选到相同的数。
- 我们猜一下结论，是不是我们这 k 次操作只合并了同一个集合。
- 然后我们发现假了，但是假的不多。假设有两个集合的最大公约数为 x 和 y ($x > y$)，我们考虑把第一个集合中最大的数剥离，将剩下的数与第二个集合直接合并，如果第一个集合中最大的数 $> 2x$ 还好说，即使剩下的数合并之后 gcd 变成 1 了都没事，多这一个 $2x$ 足够把总和拉回来了。可是如果第一个集合全是 x ，这个结论就不成立了，很难受。

- 诡异。gcd 居然和 sum 扯上关系了。
- 不难发现如果我们把 k 次操作综合起来看的话，我们实际上是将若干个集合的数合并成了一个公共 gcd。如果真的有很多很多个集合的话，这个问题就太难了，因为我们不仅要考虑怎么去选，还要考虑分开考虑每个集合的时候会不会选到相同的数。
- 我们猜一下结论，是不是我们这 k 次操作只合并了同一个集合。
- 然后我们发现假了，但是假的不多。假设有两个集合的最大公约数为 x 和 y ($x > y$)，我们考虑把第一个集合中最大的数剥离，将剩下的数与第二个集合直接合并，如果第一个集合中最大的数 $> 2x$ 还好说，即使剩下的数合并之后 gcd 变成 1 了都没事，多这一个 $2x$ 足够把总和拉回来了。可是如果第一个集合全是 x ，这个结论就不成立了，很难受。
- 不过好在，我们分析出只会有一个集合不是全相同的数，剩下的集合都只是相同的数之间的合并。我们不妨枚举相同的数合并了 l 次（这一部分的贡献是好算的，贪心选最小的就行），那不同的数就合并了 $k - l$ 次，也就是说我们需要选择 $k - l + 1$ 个不同的数进行合并。

- 对于简单版本到这里就结束了，因为值域很小，我们直接枚举合并的集合的 gcd，然后贪心选择最小的那些即可，复杂度 $O(m \ln m)$ 。但是困难版本不能这么做。

- 对于简单版本到这里就结束了，因为值域很小，我们直接枚举合并的集合的 gcd，然后贪心选择最小的那些即可，复杂度 $O(m \ln m)$ 。但是困难版本不能这么做。
- gcd 有个很好的性质就是，它的变化速度绝对比正常的值变化要小，但是 gcd 难以预测，这是它恶心的地方。考虑我们选入集合的 $k - l + 1$ 个数，如果我们将最大值改小，有哪些情况下不管 gcd 怎么变都不可能更劣。

- 对于简单版本到这里就结束了，因为值域很小，我们直接枚举合并的集合的 gcd，然后贪心选择最小的那些即可，复杂度 $O(m \ln m)$ 。但是困难版本不能这么做。
- gcd 有个很好的性质就是，它的变化速度绝对比正常的值变化要小，但是 gcd 难以预测，这是它恶心的地方。考虑我们选入集合的 $k - l + 1$ 个数，如果我们将最大值改小，有哪些情况下不管 gcd 怎么变都不可能更劣。
- 首先猜直接选最小的 $k - l + 1$ 个数显然没有任何道理，不然这连个题都不是。考虑 gcd 的基础性质：两数 gcd 不超过这两个数本身，也不超过这两个数的差。也就是说，如果我们可以把最大值改到比次大值还要小，最大值改小带来的收益绝对会比 gcd 变化带来的减益要大得多。所以我们的结论就是一定会选择最小的 $k - l$ 个数，剩下的一个数需要枚举。

- 对于简单版本到这里就结束了，因为值域很小，我们直接枚举合并的集合的 gcd，然后贪心选择最小的那些即可，复杂度 $O(m \ln m)$ 。但是困难版本不能这么做。
- gcd 有个很好的性质就是，它的变化速度绝对比正常的值变化要小，但是 gcd 难以预测，这是它恶心的地方。考虑我们选入集合的 $k - l + 1$ 个数，如果我们将最大值改小，有哪些情况下不管 gcd 怎么变都不可能更劣。
- 首先猜直接选最小的 $k - l + 1$ 个数显然没有任何道理，不然这连个题都不是。考虑 gcd 的基础性质：两数 gcd 不超过这两个数本身，也不超过这两个数的差。也就是说，如果我们可以把最大值改到比次大值还要小，最大值改小带来的收益绝对会比 gcd 变化带来的减益要大得多。所以我们的结论就是一定会选择最小的 $k - l$ 个数，剩下的一个数需要枚举。
- 可是我们前文已经枚举了一个东西了，这里再枚举又爆了。其实，我们发现我们有一个前 $k - l$ 个数的 gcd，而这玩意的变化次数只有 $O(\log a)$ 。所以我们可以枚举选择的最大值，以及枚举前 $k - l$ 个数的 gcd 是多少，然后对 l 的贡献是一个区间，然后就做完了。

题意：给定若干模式字符串 $s_1 \sim s_n$ 以及一个大字符串 t ，多组询问每次询问一个区间 (l, r) ，问 $t_l \sim t_r$ 一共出现了多少次模式串。
 $\sum |s_i| \leq 10^6, |t| \leq 5 \times 10^6$ ，询问次数 $q \leq 5 \times 10^5$ 。

- 这题会不会有很多人都做过啊 (?)

- 这题会不会有很多人都做过啊 (?)
- 首先发现如果 $l = 1, r = |t|$ 的话这题就是模板 AC 自动机，所以这题肯定不会比 AC 自动机简单。

- 这题会不会有很多人都做过啊 (?)
- 首先发现如果 $l = 1, r = |t|$ 的话这题就是模板 AC 自动机，所以这题肯定不会比 AC 自动机简单。
- 既然询问是若干个 t 的区间，那肯定会有很多可以预处理的东西，因为如果我们什么都做不了的话这个问题就相当于给了若干个毫不相关的长度总和为 2.5×10^{12} 的字符串，怎么可能做啊。也就是说，每个询问串的匹配一定可以分成两部分，一部分可以预处理掉，另一部分一定是很容易描述和查询的。

- 这题会不会有很多人都做过啊 (?)
- 首先发现如果 $l = 1, r = |t|$ 的话这题就是模板 AC 自动机，所以这题肯定不会比 AC 自动机简单。
- 既然询问是若干个 t 的区间，那肯定会有很多可以预处理的东西，因为如果我们什么都做不了的话这个问题就相当于给了若干个毫不相关的长度总和为 2.5×10^{12} 的字符串，怎么可能做啊。也就是说，每个询问串的匹配一定可以分成两部分，一部分可以预处理掉，另一部分一定是很容易描述和查询的。
- 我们考虑所有的匹配中， $t_1 \sim t_r$ 与 $t_l \sim t_r$ 的区别在哪里，因为前者是很好预处理的一个东西。首先发现，如果我们把所有匹配看成区间 $[L, R]$ ，那么 $R < l$ 的肯定没有贡献；难搞的是匹配区间满足 $L < l \leq R$ 的。假设我们找到最靠右的这样的区间 $[L, R]$ ，那么全局所有匹配区间右端点在 $[R + 1, r]$ 的都可以直接加到答案里去了，没有不合法的可能，因为都说了 R 是最靠右的。

- 先不管匹配右端点在 $[l, R]$ 这一段的答案怎么计算，我们先把这个 R 找到。

- 先不管匹配右端点在 $[l, R]$ 这一段的答案怎么计算，我们先把这个 R 找到。
- 将 t 过一遍所有 s 建成的 AC 自动机，我们可以处理出每个位置作为右端点的匹配总数，以及一个关键的信息以 i 为右端点的匹配中，最靠左的左端点在哪里，记这个值为 pre_i 。此时利用数据结构找到区间 $[l, r]$ 中最大的 i 满足 $pre_i < l$ 即可。下面我们就用 R 来代表找到的这个 i 。

- 先不管匹配右端点在 $[l, R]$ 这一段的答案怎么计算，我们先把这个 R 找到。
- 将 t 过一遍所有 s 建成的 AC 自动机，我们可以处理出每个位置作为右端点的匹配总数，以及一个关键的信息以 i 为右端点的匹配中，最靠左的左端点在哪里，记这个值为 pre_i 。此时利用数据结构找到区间 $[l, r]$ 中最大的 i 满足 $pre_i < l$ 即可。下面我们就用 R 来代表找到的这个 i 。
- 注意到 $pre_R < l \leq R$ ，我们发现 $t_l \sim t_R$ 一定是某个模式串的后缀。这明显也能预处理啊。将 s 反转再建立一个 AC 自动机，再把每个反转后的 s 进这个 AC 自动机跑一下，我们就能计算出每个模式串的后缀的匹配次数。再在维护 pre_i 的同时维护一下这个串是谁，于是我们就做完了。

- 先不管匹配右端点在 $[l, R]$ 这一段的答案怎么计算，我们先把这个 R 找到。
- 将 t 过一遍所有 s 建成的 AC 自动机，我们可以处理出每个位置作为右端点的匹配总数，以及一个关键的信息以 i 为右端点的匹配中，最靠左的左端点在哪里，记这个值为 pre_i 。此时利用数据结构找到区间 $[l, r]$ 中最大的 i 满足 $pre_i < l$ 即可。下面我们就用 R 来代表找到的这个 i 。
- 注意到 $pre_R < l \leq R$ ，我们发现 $t_l \sim t_R$ 一定是某个模式串的后缀。这明显也能预处理啊。将 s 反转再建立一个 AC 自动机，再把每个反转后的 s 进这个 AC 自动机跑一下，我们就能计算出每个模式串的后缀的匹配次数。再在维护 pre_i 的同时维护一下这个串是谁，于是我们就做完了。
- 时间复杂度 $O(|t| + \sum |s_i| + q \log |t|)$ 。印象中这玩意不是很好写。

题意：给定一个非负整数数组 a_i ，初始长度为 n ，接下来将对该数组进行 $n - 1$ 次变换，每一次变换前记数组长度为 m ，则将数组 a 变为所有 $\frac{m(m-1)}{2}$ 个形如 $a_i + a_j (i \neq j)$ 中最小的 $m - 1$ 个。

问最后剩下的数是多少。取模。

$n \leq 2 \times 10^5$ ，简单版本 $n \leq 3000$ 。

- 不是，这啥玩意。先把 a_i 排个序吧，好想，好描述。

- 不是，这啥玩意。先把 a_i 排个序吧，好想，好描述。
- 显然如果模拟的话，运算过程中不能取模，不然无法判断大小关系，但是这样的话数又太大了，完全无法存储。

- 不是，这啥玩意。先把 a_i 排个序吧，好想，好描述。
- 显然如果模拟的话，运算过程中不能取模，不然无法判断大小关系，但是这样的话数又太大了，完全无法存储。
- 一个简单 trick 是把 a_1 直接变成 0，也就是说把所有的 a_i 同时减去最小值，显然这不会影响我们比较大小关系，当然为了最后还原剩下的数，我们需要维护一个 x 表示每个数与真实值的差异，不过这个数可以取模。

- 不是，这啥玩意。先把 a_i 排个序吧，好想，好描述。
- 显然如果模拟的话，运算过程中不能取模，不然无法判断大小关系，但是这样的话数又太大了，完全无法存储。
- 一个简单 trick 是把 a_1 直接变成 0，也就是说把所有的 a_i 同时减去最小值，显然这不会影响我们比较大小关系，当然为了最后还原剩下的数，我们需要维护一个 x 表示每个数与真实值的差异，不过这个数可以取模。
- 用堆加速模拟一下，简单版本就做完了。

- 但是困难版本好像还有点距离，上面的做法显得太慢了。这时候我们就需要去分析一下性质了。

- 但是困难版本好像还有点距离，上面的做法显得太慢了。这时候我们就需要去分析一下性质了。
- 上面的做法还遗留了一些问题，也就是为什么减去最小值之后就不会超过存储范围了？手动模拟一两步就知道了。一开始 $[0, a_2, a_3, \dots, a_n]$ ，一次模拟之后，我们发现第 $n - 1$ 项不超过 $a_1 + a_n = a_n$ ，即原来的最后一项。

- 但是困难版本好像还有点距离，上面的做法显得太慢了。这时候我们就需要去分析一下性质了。
- 上面的做法还遗留了一些问题，也就是为什么减去最小值之后就不会超过存储范围了？手动模拟一两步就知道了。一开始 $[0, a_2, a_3, \dots, a_n]$ ，一次模拟之后，我们发现第 $n - 1$ 项不超过 $a_1 + a_n = a_n$ ，即原来的最后一项。
- 同时我们发现，一旦前面有一组 $a_i + a_j (i, j > 1)$ 比 a_n 小，这个 a_n 甚至连答案都无法影响了。

- 但是困难版本好像还有点距离，上面的做法显得太慢了。这时候我们就需要去分析一下性质了。
- 上面的做法还遗留了一些问题，也就是为什么减去最小值之后就不会超过存储范围了？手动模拟一两步就知道了。一开始 $[0, a_2, a_3, \dots, a_n]$ ，一次模拟之后，我们发现第 $n - 1$ 项不超过 $a_1 + a_n = a_n$ ，即原来的最后一项。
- 同时我们发现，一旦前面有一组 $a_i + a_j (i, j > 1)$ 比 a_n 小，这个 a_n 甚至连答案都无法影响了。
- 但是也是有可能有极限情况 a_n 会影响答案的，不过肯定很极限，我们分析一下。

- 若 $a_2 + a_3 > a_n$ ，则模拟一轮之后（包括减去最小值的环节），数组会变成 $[0, a_3 - a_2, \dots, a_n - a_2]$ ，看起来好像没啥事。

- 若 $a_2 + a_3 > a_n$ ，则模拟一轮之后（包括减去最小值的环节），数组会变成 $[0, a_3 - a_2, \dots, a_n - a_2]$ ，看起来好像没啥事。
- 但是如果再操作一次 a_n 还活着的话，最后一项就变成了 $a_n - a_3$ 。
 $a_2 + a_3 > a_n, a_3 \geq a_2$ ，所以 $a_n - a_3$ 比 a_n 缩小了一半以上。

- 若 $a_2 + a_3 > a_n$ ，则模拟一轮之后（包括减去最小值的环节），数组会变成 $[0, a_3 - a_2, \dots, a_n - a_2]$ ，看起来好像没啥事。
- 但是如果再操作一次 a_n 还活着的话，最后一项就变成了 $a_n - a_3$ 。
 $a_2 + a_3 > a_n, a_3 \geq a_2$ ，所以 $a_n - a_3$ 比 a_n 缩小了一半以上。
- 同时我们发现这个结论可以推广到任意的 i ，即如果连续两次操作之后某个 a_i 仍然对前 i 个数有影响的话，那么新的 a_i 必然减半，否则它将对答案没有影响。

- 若 $a_2 + a_3 > a_n$ ，则模拟一轮之后（包括减去最小值的环节），数组会变成 $[0, a_3 - a_2, \dots, a_n - a_2]$ ，看起来好像没啥事。
- 但是如果再操作一次 a_n 还活着的话，最后一项就变成了 $a_n - a_3$ 。
 $a_2 + a_3 > a_n, a_3 \geq a_2$ ，所以 $a_n - a_3$ 比 a_n 缩小了一半以上。
- 同时我们发现这个结论可以推广到任意的 i ，即如果连续两次操作之后某个 a_i 仍然对前 i 个数有影响的话，那么新的 a_i 必然减半，否则它将对答案没有影响。
- 这启示我们保留很少一部分数进行模拟。只保留约 $L = 2 \log a$ 个即可，因为若 $a_2 + a_3 \leq a_L$ 则我们能生成正确的 $a_1 \sim a_L$ ，否则两次模拟后 L 减少 2。这样就做完了。

题意：现在有一个符号完全磨损且所有键完全相同的键盘，其上共有 $m + 1$ 个键，分别为数字 $0 \sim m - 1$ 以及一个退格键（退格键在被按下时会删除目前输入的最后一个数，但如果一个数还未输入，该次按键将无效果但算作一次按键）。你想要通过这个键盘按顺序打出一个特定的长度为 n 的数组 a ，但由于在特定情况下你无法获知你需要按下的键具体为哪一个，所以你只能随机按下一个键。

已知你记忆力足够强，问在最优策略下你期望要按键多少次才能打出整个数组。常规质数模数。

$$1 \leq n \leq 10^6, 1 \leq m \leq 10^3。$$

- 首先需要思考策略。

- 首先需要思考策略。
- 假设现在下一个需要打出的数字是已知数字，直接打了；不知道下一个数字是谁，并且不知道退格是谁，那只能瞎按一个未知的键，按到退格把数字补回来（或者在开头不用管），按到正确的数字继续，按到错误的数字就乱按未知的键直到按出退格把乱打的数全删掉；不知道下一个数字是谁，且知道退格是谁，那还是只能瞎按一个未知的键，按错了就立刻删掉。

- 首先需要思考策略。
- 假设现在下一个需要打出的数字是已知数字，直接打了；不知道下一个数字是谁，并且不知道退格是谁，那只能瞎按一个未知的键，按到退格把数字补回来（或者在开头不用管），按到正确的数字继续，按到错误的数字就乱按未知的键直到按出退格把乱打的数全删掉；不知道下一个数字是谁，且知道退格是谁，那还是只能瞎按一个未知的键，按错了就立刻删掉。
- 发现这个策略里，如果遇到已知数字可以直接跳过，完全不用经历随机过程，所以我们发现数组 a 中的每一种数只有第一次出现是有意义的，后面出现只需要简单加贡献即可。所以 n 被我们直接踢出了复杂度。

- 考虑设计 dp 状态。状态里都有啥？

- 考虑设计 dp 状态。状态里都有啥？
- 首先有我们已经正确打出的有意义的数组长度 i ，其次肯定有是否已知退格键状态。

- 考虑设计 dp 状态。状态里都有啥？
- 首先有我们已经正确打出的有意义的数组长度 i ，其次肯定有是否已知退格键状态。
- 然后我们马上注意到一个严重的问题：我们乱按的过程中的那些键，具体是哪些键我怎么知道？

- 考虑设计 dp 状态。状态里都有啥？
- 首先有我们已经正确打出的有意义的数组长度 i ，其次肯定有是否已知退格键状态。
- 然后我们马上注意到一个严重的问题：我们乱按的过程中的那些键，具体是哪些键我怎么知道？
- 状压进状态里肯定没有任何搞头，我们发现这是一个概率题，我们可以将这个问题进行延迟贡献。意思是，我们只存储我们有多少键是已知但还没用的，真正到用的时候我们再通过概率去分配这个键到底是不是我们想要的。

- 经过反复的修改，我们可以 不那么 轻松地得出一个 dp 状态：
 $dp_{i,j,0/1/2/3}$ 表示正确打出的有意义数组长度为 i ，已知但还没有用过的键的数量为 j ，以及一个状态。

- 经过反复的修改，我们可以 不那么 轻松地得出一个 dp 状态：
 $dp_{i,j,0/1/2/3}$ 表示正确打出的有意义数组长度为 i ，已知但还没有用过的键的数量为 j ，以及一个状态。
 - 0 表示运气超群打的全对，并且不知道退格键是谁；

- 经过反复的修改，我们可以 不那么 轻松地得出一个 dp 状态：
 $dp_{i,j,0/1/2/3}$ 表示正确打出的有意义数组长度为 i ，已知但还没有用过的键的数量为 j ，以及一个状态。
 - 0 表示运气超群打的全对，并且不知道退格键是谁；
 - 1 表示已经打错了，但是还不知道退格键是谁；

- 经过反复的修改，我们可以 不那么 轻松地得出一个 dp 状态：
 $dp_{i,j,0/1/2/3}$ 表示正确打出的有意义数组长度为 i ，已知但还没有用过的键的数量为 j ，以及一个状态。
 - 0 表示运气超群打的全对，并且不知道退格键是谁；
 - 1 表示已经打错了，但是还不知道退格键是谁；
 - 2 表示已经知道了退格键是谁，并且还没有考虑那 j 个键里有没有我们接下来想要的键；

- 经过反复的修改，我们可以 不那么 轻松地得出一个 dp 状态：
 $dp_{i,j,0/1/2/3}$ 表示正确打出的有意义数组长度为 i ，已知但还没有用过的键的数量为 j ，以及一个状态。
 - 0 表示运气超群打的全对，并且不知道退格键是谁；
 - 1 表示已经打错了，但是还不知道退格键是谁；
 - 2 表示已经知道了退格键是谁，并且还没有考虑那 j 个键里有没有我们接下来想要的键；
 - 3 表示已经知道了退格键是谁，并且我们发现那 j 个键里真的没有我们想要的键。

- 经过反复的修改，我们可以 不那么 轻松地得出一个 dp 状态：
 $dp_{i,j,0/1/2/3}$ 表示正确打出的有意义数组长度为 i ，已知但还没有用过的键的数量为 j ，以及一个状态。
 - 0 表示运气超群打的全对，并且不知道退格键是谁；
 - 1 表示已经打错了，但是还不知道退格键是谁；
 - 2 表示已经知道了退格键是谁，并且还没有考虑那 j 个键里有没有我们接下来想要的键；
 - 3 表示已经知道了退格键是谁，并且我们发现那 j 个键里真的没有我们想要的键。
- 在这种情况下，后续想要要把整个序列打出来所需要的期望步数。答案直接取用 $f_{0,0,0}$ 即可。

- 考虑转移。

- 考虑转移。
- 0 状态随机打一个，如果是退格键立即补回来，走到 $f_{i,j,2}$ ；是正确的键，走到 $f_{i+1,j,0}$ ；不是正确的键，提前把删除时的贡献算好，走到 $f_{i,j+1,1}$ ；

- 考虑转移。
- 0 状态随机打一个，如果是退格键立即补回来，走到 $f_{i,j,2}$ ；是正确的键，走到 $f_{i+1,j,0}$ ；不是正确的键，提前把删除时的贡献算好，走到 $f_{i,j+1,1}$ ；
- 1 状态随机打一个，是退格键没代价，走到 $f_{i,j,1}$ ；不是退格键，提前把删除时的贡献算好，走到 $f_{i,j+1,1}$ ；

- 考虑转移。
- 0 状态随机打一个，如果是退格键立即补回来，走到 $f_{i,j,2}$ ；是正确的键，走到 $f_{i+1,j,0}$ ；不是正确的键，提前把删除时的贡献算好，走到 $f_{i,j+1,1}$ ；
- 1 状态随机打一个，是退格键没代价，走到 $f_{i,j,1}$ ；不是退格键，提前把删除时的贡献算好，走到 $f_{i,j+1,1}$ ；
- 2 状态是一个中间态，这时我们不是要打一个键，而是去检查之前的 j 个键有没有我们想要的。由于之前敲的时候我们只知道它不是我们想要的，这时候我们去观测一下它，让它坍塌，接下来要打的键有一定概率在这 j 个键里。在 j 个键里，走到 $f_{i+1,j-1,2}$ ；不在 j 个键里，走到 $f_{i,j,3}$ ；

- 考虑转移。
- 0 状态随机打一个，如果是退格键立即补回来，走到 $f_{i,j,2}$ ；是正确的键，走到 $f_{i+1,j,0}$ ；不是正确的键，提前把删除时的贡献算好，走到 $f_{i,j+1,1}$ ；
- 1 状态随机打一个，是退格键没代价，走到 $f_{i,j,1}$ ；不是退格键，提前把删除时的贡献算好，走到 $f_{i,j+1,1}$ ；
- 2 状态是一个中间态，这时我们不是要打一个键，而是去检查之前的 j 个键有没有我们想要的。由于之前敲的时候我们只知道它不是我们想要的，这时候我们去观测一下它，让它坍塌，接下来要打的键有一定概率在这 j 个键里。在 j 个键里，走到 $f_{i+1,j-1,2}$ ；不在 j 个键里，走到 $f_{i,j,3}$ ；
- 3 状态随机打一个，不是正确的键，立即退格删除它，走到 $f_{i,j+1,3}$ ；是正确的键，走到 $f_{i+1,j,2}$ 。

- 考虑转移。
- 0 状态随机打一个，如果是退格键立即补回来，走到 $f_{i,j,2}$ ；是正确的键，走到 $f_{i+1,j,0}$ ；不是正确的键，提前把删除时的贡献算好，走到 $f_{i,j+1,1}$ ；
- 1 状态随机打一个，是退格键没代价，走到 $f_{i,j,1}$ ；不是退格键，提前把删除时的贡献算好，走到 $f_{i,j+1,1}$ ；
- 2 状态是一个中间态，这时我们不是要打一个键，而是去检查之前的 j 个键有没有我们想要的。由于之前敲的时候我们只知道它不是我们想要的，这时候我们去观测一下它，让它坍塌，接下来要打的键有一定概率在这 j 个键里。在 j 个键里，走到 $f_{i+1,j-1,2}$ ；不在 j 个键里，走到 $f_{i,j,3}$ ；
- 3 状态随机打一个，不是正确的键，立即退格删除它，走到 $f_{i,j+1,3}$ ；是正确的键，走到 $f_{i+1,j,2}$ 。
- 概率系数略过，如果把 dp 理解为一类情况的集合，2 状态在干的事情就是说：我这里非常精确的有一部分是已知下个键的，一部分是未知的，且这个比例是一个可计算的式子，这样理解也是对的。然后就做完了。

题意：交互。交互库有一个 n 个变量的函数，仅由变量数量不限的 $\min, \max$ 组成，且保证按照 $x_1 \sim x_n$ 顺序可以写出该函数，且每个 x_i 仅出现一次。你每次可以给出一组变量，交互库会返回函数值，你需要在不超过 C 次询问后精准猜出这个函数是什么，任意与原函数等价的函数都可以（因为测试方法是给你若干组变量让你返回结果）。

H1: $n \leq 70, C = 10000$;

H2: $n \leq 200, C = 10000$;

H3: $n \leq 400, C = 1500$ 。

- 不是人啊。H1/2 做法和 H3 关系不大，但我觉得可以讲一下。

- 不是人啊。H1/2 做法和 H3 关系不大，但我觉得可以讲一下。
- 先全按 01 输入做一下看看，这样 $\max$ 就是或， $\min$ 就是与。

- 不是人啊。H1/2 做法和 H3 关系不大，但我觉得可以讲一下。
- 先全按 01 输入做一下看看，这样 $\max$ 就是或， $\min$ 就是与。
- 如果我们把这个函数看作一个树形结构的话，相邻的同种操作节点就可以直接拆开来，这样相邻的节点肯定是异种操作。

- 不是人啊。H1/2 做法和 H3 关系不大，但我觉得可以讲一下。
- 先全按 01 输入做一下看看，这样 $\max$ 就是或， $\min$ 就是与。
- 如果我们把这个函数看作一个树形结构的话，相邻的同种操作节点就可以直接拆开来，这样相邻的节点肯定是异种操作。
- 首先我们需要先确定根节点是什么类型，如果是或的话，只需要一个子节点成为 1 就可以了，与的话需要所有子节点都是 1。我们考虑这样两组自变量：前缀 l 个 1，其余均为 0；以及后缀从 r 开始全是 1，其余均为 0。找到最小的 l 和最大的 r ，根节点是或节点当且仅当 $l < r$ ，原因是或根只需要一个子节点是 1，故 l 肯定不超过第一个子节点的变量数， $r \sim n$ 不超过最后一个子节点的范围，与需要 l 触及最后一个子节点 r 触及第一个子节点。

- 不是人啊。H1/2 做法和 H3 关系不大，但我觉得可以讲一下。
- 先全按 01 输入做一下看看，这样 $\max$ 就是或， $\min$ 就是与。
- 如果我们把这个函数看作一个树形结构的话，相邻的同种操作节点就可以直接拆开来，这样相邻的节点肯定是异种操作。
- 首先我们需要先确定根节点是什么类型，如果是或的话，只需要一个子节点成为 1 就可以了，与的话需要所有子节点都是 1。我们考虑这样两组自变量：前缀 l 个 1，其余均为 0；以及后缀从 r 开始全是 1，其余均为 0。找到最小的 l 和最大的 r ，根节点是或节点当且仅当 $l < r$ ，原因是或根只需要一个子节点是 1，故 l 肯定不超过第一个子节点的变量数， $r \sim n$ 不超过最后一个子节点的范围，与需要 l 触及最后一个子节点 r 触及第一个子节点。
- 如果根是与节点，那么翻转 01 后就变成了或节点，我们接下来只讨论或根。

- 不是人啊。H1/2 做法和 H3 关系不大，但我觉得可以讲一下。
- 先全按 01 输入做一下看看，这样 $\max$ 就是或， $\min$ 就是与。
- 如果我们把这个函数看作一个树形结构的话，相邻的同种操作节点就可以直接拆开来，这样相邻的节点肯定是异种操作。
- 首先我们需要先确定根节点是什么类型，如果是或的话，只需要一个子节点成为 1 就可以了，与的话需要所有子节点都是 1。我们考虑这样两组自变量：前缀 l 个 1，其余均为 0；以及后缀从 r 开始全是 1，其余均为 0。找到最小的 l 和最大的 r ，根节点是或节点当且仅当 $l < r$ ，原因是或根只需要一个子节点是 1，故 l 肯定不超过第一个子节点的变量数， $r \sim n$ 不超过最后一个子节点的范围，与需要 l 触及最后一个子节点 r 触及第一个子节点。
- 如果根是与节点，那么翻转 01 后就变成了或节点，我们接下来只讨论或根。
- 我们需要将树划分为两个区域来递归求解。

- 考虑找到第一个子树的右边界。注意不能直接使用刚刚的 l ，因为 l 可能只是让第三层最后一个或节点变成 1 了而已。

- 考虑找到第一个子树的右边界。注意不能直接使用刚刚的 l ，因为 l 可能只是让第三层最后一个或节点变成 1 了而已。
- 不过我们注意到，如果保证 $1 \sim l$ 都是 0，那么第一个子树就一定为 0 了。由于第二层是与节点， $1 \sim l$ 必然已经触及到了第三层的最后一个子树，所以强制 $1 \sim l$ 都是 0，就保证了前面的第三层子树一定全为 0。

- 考虑找到第一个子树的右边界。注意不能直接使用刚刚的 l ，因为 l 可能只是让第三层最后一个或节点变成 1 了而已。
- 不过我们注意到，如果保证 $1 \sim l$ 都是 0，那么第一个子树就一定为 0 了。由于第二层是与节点， $1 \sim l$ 必然已经触及到了第三层的最后一个子树，所以强制 $1 \sim l$ 都是 0，就保证了前面的第三层子树一定全为 0。
- 此时从 $l+1$ 开始往后再赋值 1，设最少赋值到了 k 再次满足函数值为 1，那此时为 1 的就是第二棵子树了。我们干这样一个惊天动地的事：我们考虑将前 $k-1$ 个位置赋值 1。这样我们就能保证第二棵子树一定是 0（因为 k 是第一个让第二棵子树变成 1 的位置），而第一课子树一定是 1（ $k \geq l+1$ ，有 $k-1 \geq l$ ）。

- 从 $i = 1$ 开始遍历到 $k - 1$ ，将第 i 个位置赋为 0 重新询问，如果函数值仍为 1 就让这个位置永久变为 0 了，否则恢复它为 1。这样做的好处是：对于第二层第二棵子树，所有 1 都会被清零；对于第二层第一棵子树下面的任意一个子树，若其为与节点，则所有子节点都为 1，若其为或节点，一定只有最右边的子节点为 1。也就是说，每个子树都留下了最靠右的一条链，我们成功得到了第二层第一棵子树的右边界！设其为 z ，此时我们划分为 $f = \max(f(x_1 \sim x_z), f(x_{z+1} \sim x_n))$ ，递归求解即可做到 $2n^2$ 次询问。

- 从 $i = 1$ 开始遍历到 $k - 1$ ，将第 i 个位置赋为 0 重新询问，如果函数值仍为 1 就让这个位置永久变为 0 了，否则恢复它为 1。这样做的好处是：对于第二层第二棵子树，所有 1 都会被清零；对于第二层第一棵子树下面的任意一个子树，若其为与节点，则所有子节点都为 1，若其为或节点，一定只有最右边的子节点为 1。也就是说，每个子树都留下了最靠右的一条链，我们成功得到了第二层第一棵子树的右边界！设其为 z ，此时我们划分为 $f = \max(f(x_1 \sim x_z), f(x_{z+1} \sim x_n))$ ，递归求解即可做到 $2n^2$ 次询问。
- 题解声称这玩意可以通过 “simulating from both ends” 来做到 $O(n \log n)$ ，但是完全没写具体怎么做。我觉得这不 trivial，但我的思维能力已经消失了，所以这里就不展开了。

- 接下来我们考虑 H3 怎么做。设原树为 T 。

- 接下来我们考虑 H3 怎么做。设原树为 T 。
- 发现我们一直都是在用 01 序列做询问，我们不妨用正常的 $p_i = i$ 问一遍看看怎么个事。

- 接下来我们考虑 H3 怎么做。设原树为 T 。
- 发现我们一直都是在用 01 序列做询问，我们不妨用正常的 $p_i = i$ 问一遍看看怎么个事。
- 假设返回值是 x ，这说明我们从根节点一路往下，遇到 $\max$ 走最右侧子树，遇到 $\min$ 走最左侧子树，最后走到了 x 。

- 接下来我们考虑 H3 怎么做。设原树为 T 。
- 发现我们一直都是在用 01 序列做询问，我们不妨用正常的 $p_i = i$ 问一遍看看怎么个事。
- 假设返回值是 x ，这说明我们从根节点一路往下，遇到 $\max$ 走最右侧子树，遇到 $\min$ 走最左侧子树，最后走到了 x 。
- 此时如果我们将整个序列改为 $p_i = x_i + [i > x]\infty$ ，也就是从 $x + 1$ 开始全为无穷，由于该函数一定为有界函数，所以所有与无穷有关的变量我们可以直接扔掉，这样我们的树可以化简，得到一棵仅使用 x 个变量的 L 树，其根节点为 $\max$ ；同理，我们令 $p_i = x_i - [i < x]\infty$ ，得到一棵仅使用 $n - x + 1$ 个变量的 R 树，其根节点为 $\min$ 。 L 和 R 有一个共同点，就是第 x 个变量一定是直接挂在根节点上的。

- 假设我们能够获知 $L \setminus \{x\}$ 和 $R \setminus \{x\}$ 的结构，我们考虑我们该如何合并它。首先，设 $L \setminus \{x\}$ 第二层最右侧的子树为 A ， $R \setminus \{x\}$ 最左侧的子树为 B ，我们需要知道在 T 中是先计算 $\max\{A, x\}$ 还是先计算 $\min\{x, B\}$ （如果是多个参数，我们可以视为多个二元运算的复合）。我们非常聪明地：

- 假设我们能够获知 $L \setminus \{x\}$ 和 $R \setminus \{x\}$ 的结构，我们考虑我们该如何合并它。首先，设 $L \setminus \{x\}$ 第二层最右侧的子树为 A ， $R \setminus \{x\}$ 最左侧的子树为 B ，我们需要知道在 T 中是先计算 $\max\{A, x\}$ 还是先计算 $\min\{x, B\}$ （如果是多个参数，我们可以视为多个二元运算的复合）。我们非常聪明地：
 - 将 A 左侧的所有点设为 1；

- 假设我们能够获知 $L \setminus \{x\}$ 和 $R \setminus \{x\}$ 的结构，我们考虑我们该如何合并它。首先，设 $L \setminus \{x\}$ 第二层最右侧的子树为 A ， $R \setminus \{x\}$ 最左侧的子树为 B ，我们需要知道在 T 中是先计算 $\max\{A, x\}$ 还是先计算 $\min\{x, B\}$ （如果是多个参数，我们可以视为多个二元运算的复合）。我们非常聪明地：
 - 将 A 左侧的所有点设为 1；
 - 将 A 内点设为 4；

- 假设我们能够获知 $L \setminus \{x\}$ 和 $R \setminus \{x\}$ 的结构，我们考虑我们该如何合并它。首先，设 $L \setminus \{x\}$ 第二层最右侧的子树为 A ， $R \setminus \{x\}$ 最左侧的子树为 B ，我们需要知道在 T 中是先计算 $\max\{A, x\}$ 还是先计算 $\min\{x, B\}$ （如果是多个参数，我们可以视为多个二元运算的复合）。我们非常聪明地：
 - 将 A 左侧的所有点设为 1；
 - 将 A 内点设为 4；
 - 将 x 设为 3；

- 假设我们能够获知 $L \setminus \{x\}$ 和 $R \setminus \{x\}$ 的结构，我们考虑我们该如何合并它。首先，设 $L \setminus \{x\}$ 第二层最右侧的子树为 A ， $R \setminus \{x\}$ 最左侧的子树为 B ，我们需要知道在 T 中是先计算 $\max\{A, x\}$ 还是先计算 $\min\{x, B\}$ （如果是多个参数，我们可以视为多个二元运算的复合）。我们非常聪明地：
 - 将 A 左侧的所有点设为 1；
 - 将 A 内点设为 4；
 - 将 x 设为 3；
 - 将 B 内点设为 2；

- 假设我们能够获知 $L \setminus \{x\}$ 和 $R \setminus \{x\}$ 的结构，我们考虑我们该如何合并它。首先，设 $L \setminus \{x\}$ 第二层最右侧的子树为 A ， $R \setminus \{x\}$ 最左侧的子树为 B ，我们需要知道在 T 中是先计算 $\max\{A, x\}$ 还是先计算 $\min\{x, B\}$ （如果是多个参数，我们可以视为多个二元运算的复合）。我们非常聪明地：
 - 将 A 左侧的所有点设为 1；
 - 将 A 内点设为 4；
 - 将 x 设为 3；
 - 将 B 内点设为 2；
 - 将 B 右侧点设为 5。

- 假设我们能够获知 $L \setminus \{x\}$ 和 $R \setminus \{x\}$ 的结构，我们考虑我们该如何合并它。首先，设 $L \setminus \{x\}$ 第二层最右侧的子树为 A ， $R \setminus \{x\}$ 最左侧的子树为 B ，我们需要知道在 T 中是先计算 $\max\{A, x\}$ 还是先计算 $\min\{x, B\}$ （如果是多个参数，我们可以视为多个二元运算的复合）。我们非常聪明地：
 - 将 A 左侧的所有点设为 1；
 - 将 A 内点设为 4；
 - 将 x 设为 3；
 - 将 B 内点设为 2；
 - 将 B 右侧点设为 5。
- 如果返回了 4，说明先计算 $\min\{x, B\}$ ，将 B 从 R 中删除，并将 x 变为 $\min\{x, B\}$ ；同理，若返回了 2，将 A 从 L 中删除，并将 x 变为 $\max\{A, x\}$ ，然后重复这个算法（注意后续 x 内有不只一个点，需将 x 内所有点均设为 3）直到 L, R 均为空，此时就得到了 T 。

- 假设我们能够获知 $L \setminus \{x\}$ 和 $R \setminus \{x\}$ 的结构，我们考虑我们该如何合并它。首先，设 $L \setminus \{x\}$ 第二层最右侧的子树为 A ， $R \setminus \{x\}$ 最左侧的子树为 B ，我们需要知道在 T 中是先计算 $\max\{A, x\}$ 还是先计算 $\min\{x, B\}$ （如果是多个参数，我们可以视为多个二元运算的复合）。我们非常聪明地：
 - 将 A 左侧的所有点设为 1；
 - 将 A 内点设为 4；
 - 将 x 设为 3；
 - 将 B 内点设为 2；
 - 将 B 右侧点设为 5。
- 如果返回了 4，说明先计算 $\min\{x, B\}$ ，将 B 从 R 中删除，并将 x 变为 $\min\{x, B\}$ ；同理，若返回了 2，将 A 从 L 中删除，并将 x 变为 $\max\{A, x\}$ ，然后重复这个算法（注意后续 x 内有不只一个点，需将 x 内所有点均设为 3）直到 L, R 均为空，此时就得到了 T 。
- 所以我们在一开始的时候找到 x 并分治下去，之后在这里合并就做完了。

- 接下来分析询问次数。

- 接下来分析询问次数。
- 设 L 有 a 个第二层节点， R 有 b 个第二层节点，设 $H(T)$ 为重新构建 T 这棵树所需的询问次数，则
$$H(T) \leq H(L) + H(R) + a + b。$$

- 接下来分析询问次数。
- 设 L 有 a 个第二层节点， R 有 b 个第二层节点，设 $H(T)$ 为重新构建 T 这棵树所需的询问次数，则
$$H(T) \leq H(L) + H(R) + a + b。$$
- 由于 L 在计算时，其 x 必落在最右侧第二层节点，而这会导致分治时最右侧子树每次与 L 的左侧 $a - 1$ 个子树合并时，都已没有右半部分节点可以进行合并，故无需再次查询，查询次数等价于其没有和 L 其他子树合并； R 同理。

- 接下来分析询问次数。
- 设 L 有 a 个第二层节点， R 有 b 个第二层节点，设 $H(T)$ 为重新构建 T 这棵树所需的询问次数，则
$$H(T) \leq H(L) + H(R) + a + b。$$
- 由于 L 在计算时，其 x 必落在最右侧第二层节点，而这会导致分治时最右侧子树每次与 L 的左侧 $a - 1$ 个子树合并时，都已没有右半部分节点可以进行合并，故无需再次查询，查询次数等价于其没有和 L 其他子树合并； R 同理。
- 故 $H(T) \leq \sum H(L_i) + \sum H(R_i) + a + b$ ，容易证明
$$H(T) \leq 3|T| - 2，$$
 则可以在 1198 次询问内解决此题。